



Exercise 4

The objective of this exercise is to understand, practice and apply the concept of model predictive control with reference tracking and long prediction horizons to a three-phase motor drive system. To solve the underlying integer optimization problem, the notion of sphere decoding will be used.

4.1 Model predictive control with reference tracking: Implementation

Consider the three-phase inverter drive system shown in Fig. 1. A three-level neutral-point-clamped (NPC) converter with the total dc-link voltage V_{dc} is connected to a squirrel-cage induction machine. The neutral point potential is assumed to be zero. The rated machine values are provided in Table 1. The system parameters are summarized in Table 2.

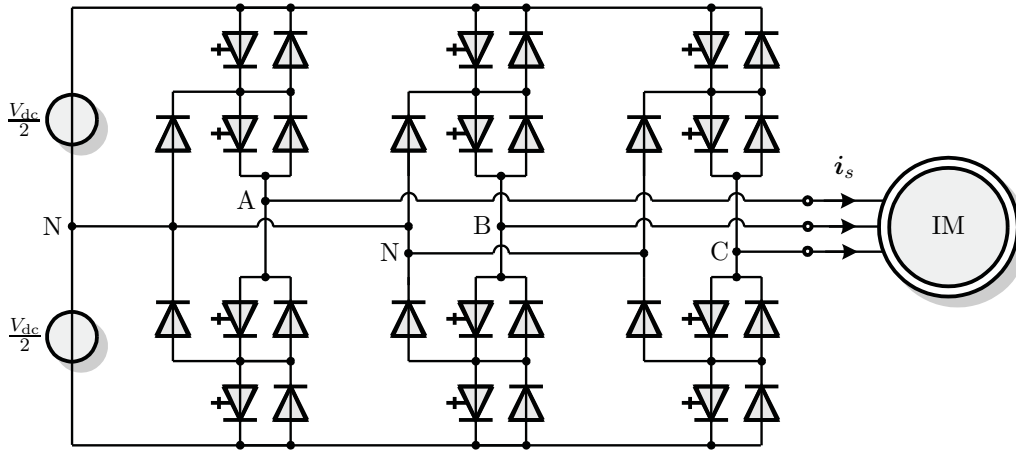


Figure 1: Three-level three-phase neutral point clamped inverter driving an induction machine with a fixed neutral point potential

Table 1: Rated values of the induction machine

Parameter	Symbol	SI value
Voltage	V_R	3300 V
Current	I_R	356 A
Apparent power	S_R	2.035 MVA
Angular stator frequency	ω_{sR}	$2\pi 50$ rad/s

1. Preparation: Download the Matlab/Simulink files from the course webpage. Start Matlab and open the Simulink model `drive.slx`. A few comments are provided hereafter:

- The initialization file `driveIni.m` is called when the simulation is started, see **Model Settings / Model Properties / Callbacks / InitFcn**. The parameter and controller settings are provided in this file. The initialization file calls the file `machModel.m` in order to derive the discrete-time machine model in the per unit system and the stationary orthogonal reference

Table 2: Drive parameters in the SI (left) and per unit (p.u.) system (right) of the three-level NPC inverter drive system

Parameter	Symbol	SI value	Symbol	p.u. value
Stator resistance	R_s	57.61 m Ω	R_s	0.0108 p.u.
Rotor resistance	R_r	48.89 m Ω	R_r	0.0091 p.u.
Stator leakage inductance	L_{ls}	2.544 mH	X_{ls}	0.1493 p.u.
Rotor leakage inductance	L_{lr}	1.881 mH	X_{lr}	0.1104 p.u.
Main inductance	L_m	40.01 mH	X_m	2.349 p.u.
Number of pole pairs	p	5		
Dc-link voltage	V_{dc}	5.2 kV	V_{dc}	1.930 p.u.

frame. The corresponding continuous-time machine model is provided in (2.59) and (2.60) of the book. Note that the time axis has been normalized, too. To simplify the exercise, we consider operation at a fixed angular rotor speed ω_r . The initialization file also derives the matrices of the optimization problem with the help of `problMatrices.m`.

- The yellow block, which executes the current controller (Matlab function named `currCtrl`), needs to be completed. The current controller calls the function `sphDec` (see file `sphDec.m`), which executes the sphere decoder.
 - The drive system is simulated at the short sampling interval of $2.5 \mu\text{s}$, while the controller is run at the (often much) longer sampling interval $T_s > 2.5 \mu\text{s}$. To achieve this, several rate transition blocks are used in the Simulink model.
 - We distinguish between continuous-time quantities, which depend on the time t , and discrete-time quantities, which are a function of the time step k .
 - Unlike in previous exercises, the SimPowerSystems toolbox is not used. Instead, the machine model has been implemented manually in Simulink to have full control over the initial state. The converter has been reduced to a gain, i.e., to $0.5V_{dc}$.
2. Complete the sphere decoder in the file `sphDec.m`. Neglect the switching constraint (that prevents switching between 1 and -1). Test the sphere decoder using the script `debug_sphDec.m`. What is the optimal sequence of switch positions for this particular problem instance assuming the prediction horizon $N_p = 2$?
 3. How many candidate switching sequences are explored in this problem instance? Assume that the optimal switching sequence at time step $k - 1$ was $\mathbf{U}^*(k - 1) \equiv \mathbf{U}(k - 1) = [0 \ 1 \ -1 \ 0 \ 1 \ 0]^T$. What is a good initial solution for the sphere decoder? How many candidate switching sequences are then explored?
 4. Complete function `currCtrl` that implements the current controller. Run the drive system with $T_s = 100 \mu\text{s}$, the penalty on switching $\lambda_u = 5 \cdot 10^{-3}$ and the prediction horizon $N_p = 2$. What is the average switching frequency per device?
 5. Does the sphere decoder find the optimal solution quickly? Is it possible to reduce the computation time by providing a better initial solution, e.g., the previous switching sequence shifted by one time step? Implement the improved initial solution in `currCtrl` and test it.
 6. Apply step changes in the torque by manipulating the current reference in the rotating reference frame accordingly, i.e., its q -component. Consider a step change to $T_{e,\text{ref}} = 0$ p.u. ($i_{sq,\text{ref}} = 0$ p.u.). Note that the current references are defined in the lower left-hand corner of the Simulink model `drive.slx`.

4.2 Model predictive control with reference tracking: Extensions

After having implemented and tested the current controller, we will now extend it by considering a longer prediction horizon, adding the switching constraint and considering an alternative method to determine an initial solution.

1. Extend the prediction horizon to $N_p = 3$ in `problMatrices.m`. Do not forget to set $N_p = 3$ in `drivelni.m`. Ensure that you use an appropriate initial solution $\mathbf{U}_{\text{ini}}(k)$. Test the controller for one problem instance using the script `debug_sphDec.m`. What is the optimal sequence of switch positions for this particular problem instance?
2. Run the drive system with $T_s = 100 \mu\text{s}$, the penalty on switching $\lambda_u = 5 \cdot 10^{-3}$ and the prediction horizon $N_p = 3$. What is the average switching frequency per device? How many candidate switching sequences are considered on average when using the initial solution as in Exercise 4.1.5?
3. Add the switching constraint to the sphere decoder. Does the behavior of the controller change during steady-state operation? Does it change during torque steps?
4. An alternative way to find an initial solution is the so called *Babai* estimate. The Babai estimate is obtained by rounding the unconstrained solution and saturating it at ± 1 to ensure that the integer switching sequence is bounded between 1 and -1 . Add the Babai estimate to the current controller in `currCtrl`. Consider removing the switching constraint from the sphere decoder.
5. Consider the case when the Babai estimate is used as initial condition and the switching constraint is imposed in the sphere decoder. Will the sphere decoder always find a solution, particularly in the presence of transients?